IN3050 Oblig 1

Name: Zhen Wei Liu
Username: zhenwl

Date: February 2025

Contents

1 Introduction **2**

2 Exhaustive search **2**
2.1 Possible routes . . . 2
2.2 Results from Exhaustive search . . . 2

3 Hill climbing **4**
3.1 Idea . . . 4
3.2 Results from Hill climbing . . . 5

4 Genetic algorithm **7**
4.1 Mutation . . . 7
4.2 Crossover . . . 7
4.3 Selection . . . 7
4.4 Population size . . . 8
4.5 Fitness . . . 8
4.6 Termination condition . . . 8
4.7 Results from Genetic Algorithm . . . 8
4.8 Observations . . . 15

5 References and files **16**

1 Introduction

The main goal of the traveling salesman problem (TSP) is to find the optimal route that visits each city once before returning to the start, given n cities. There are several methods to find the optimal route (or the shortest route as well). The efficiency of those methods mainly depends on how many cities we have. The more cities, the harder it is to find the optimal solution.

To start, we first scan the .csv-file to create a table similar to the example given in the assignment. It means that if we have n cities, then our table of distances should have a size $(n+1) \times n$. The first row contains the name of these n cities.

The instructions for results from each section will be mentioned at the end of each section.

2 Exhaustive search

2.1 Possible routes

With exhaustive search you are looking at every possible route to find the shortest path. This method is guaranteed to find the best solution, but it can be quite computationally heavy when n is large. The complexity of exhaustive search is $O(n!)$, which is terrible. This is due to the fact that for n cities the number of permutations is $n!$. Then we have to check all these permutations to find the shortest tour. Thus, we can see that exhaustive search is quite computationally heavy.

2.2 Results from Exhaustive search

In order to get accurate results for the time it takes to find the shortest tour for 6 to 10 cities we can calculate the average time for each number of cities. Here I have chosen the average of 10 runs for each time. And thus the following plot

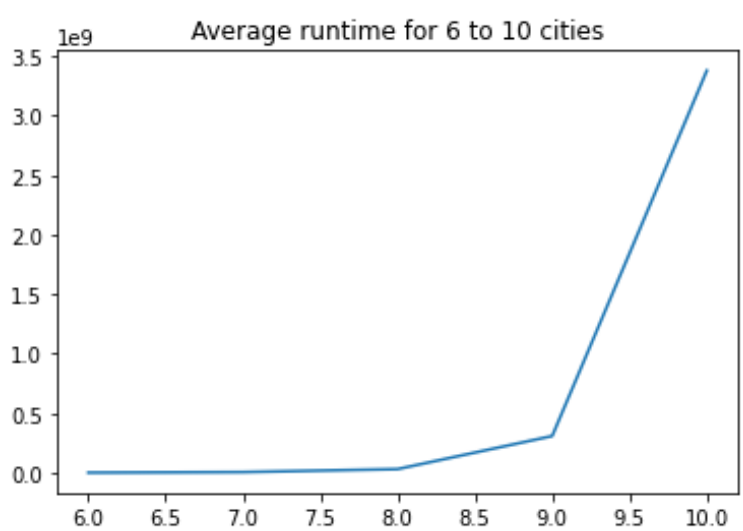

Figure 1: Exhaustive search

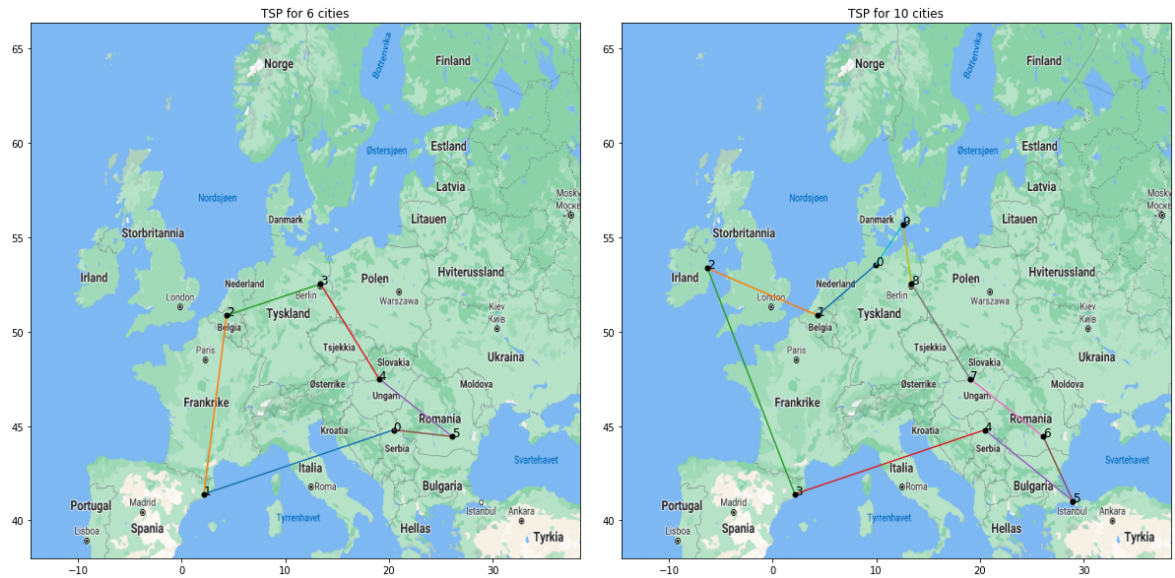

Figure 2: Exhaustive search results. Left: n = 6. Right: n = 10

For the first 10 cities, the shortest tour took around 3 seconds to find. Based on this we can sort of estimate how much it will take to search in 24 cities. We know that the total amount of permutations for n cities is $n!$. Thus, when $n = 24$ we get

$$\begin{aligned} 24! &= 24 \cdot 23 \cdot 22 \cdot ... \cdot 11 \cdot 10! \\ &= \frac{24!}{10!} \cdot 10! \end{aligned}$$

If $O(10!)$ has a time usage of 3 seconds, then for $n = 24$ we roughly get

$$2 \cdot \frac{24!}{10!} \cdot 3 \quad s = 3.23 \cdot 10^{10} \quad years$$

Which is above the age of the universe. Thus, for 24 cities, its already near impossible. The reason for multiplying with 2 is due to the fact that it requires O($n!$) to create all the permutations, then O($n!$) to find all the distances.
And here are the distances for $n = 6$ and $n = 10$:

```
>>> IN3050_Oblig1.py

Exhaustive search
---- Number of cities:  6  ----
Shortest distance:  5018.81  km
['Belgrade', 'Barcelona', 'Brussels', 'Berlin', 'Budapest', 'Bucharest']


---- Number of cities:  10  ----
Shortest distance:  7486.31  km
['Hamburg', 'Brussels', 'Dublin', 'Barcelona', 'Belgrade', 'Istanbul', 'Bucharest', 'Budapest', 'Berl
```

The results are from line 36-74 of **IN3050_Oblig1.py**.

3 Hill climbing

3.1 Idea

To find the optimal solutions we can use hill climbing. The idea is to assume you have a landscape of undiscovered solutions to perform a local search on. You start by choosing a random solution and perform local searches until you find a peak (the optimal solution) in this landscape. To perform the local search we first need to find it's neighbors. There are many ways to find it's "neighbors". One way is to make small "tweaks" to our solution. By small tweaks, we mean swapping two cities in our solution. Then we find the current best neighboring solution (steepest hill) and find it's best neighbor and so on until we can't find any better.

In the TSP-problem, our choice of neighbors is by making small tweaks - swapping two cities. By making these small tweaks, we keep the tour distance roughly the same, with some small deviations. We take the last city and swaps it with one of it's previous different cities, then we move down to the second last city to do the same until we reach our first and second cities. Here are some example:

Consider 5 cities in a list: [1,2,3,4,5]
Then it has these 10 neighbors: [1,2,3,5,4], [1,2,5,4,3], [1,5,3,4,2], [5,2,3,4,1], [1,2,4,3,5], [1,4,3,2,5], [4,2,3,1,5], [1,3,2,4,5], [3,2,1,4,5], [2,1,3,4,5]

And for 4 cities in a list: [1,2,3,4]
it has these 6 neighbors: [1,2,4,3], [1,4,3,2], [4,2,3,1], [1,3,2,4], [3,2,1,4], [2,1,3,4]

Thus, the mathematics says that for a solution with n cities, we will have
$n(n-1)/2$ neighbors with this method. This means that the size of our local search space depends on n. The larger n is, the more neighbors. Another reason for implementing this choice of neighbors is that it's sufficient in finding the neighbors, and since we know it's n cities it's easy to predict how many neighbors we will find. This choice of neighbors is implemented in $def\ generateNeighbors()$ of **IN3050_Oblig1.py**.

Obviously, other choices of neighbors exists. More neighbors means a larger local search, which might lead to better solutions being discovered, at the cost of computational power. This matters since hill climbing is a greedy algorithm. It only chooses the best neighbor in each instance of neighbors, without regards for what might exist with further exploration. This results in finding a local optimum without knowing if it is a global minimum, hence the variations in our results. In addition, the initial solution we start with is always random, hence there is no guarantee that we can find the global optimum (best solution).

In order to check how well hill climbing performs compared to exhaustive search, we find the best, worst and mean tour of 20 runs. In addition, we also find the standard deviation for 10 and 24 cities. The standard deviation is an indicator of the stability of the mean distance. A low standard deviation tells us that the result after each run is close to the mean distance. The standard deviation can be formulated as such:

$$\sigma_n^2 = \frac{1}{20}\sum_{i=1}^{20}(d_i^n - \mu_n)^2$$

$$\sigma_n = \sqrt{\sigma_n^2}$$

where n is the number of cities, μ_n is the mean distance for n cities, d_i^n is the best tour distance for n cities after the i-th run, σ_n^2 is the variance and σ_n is the standard deviation for n cities.

3.2 Results from Hill climbing

Here are the output results after 20 runs for both 10 and 24 cities:

```
    >>> IN3050_Oblig1.py

Reloaded modules: IN3050_Oblig1_functions
Hill climbing
---- Number of cities:  10  ----
Best solution:  7486.31  km
['Istanbul', 'Belgrade', 'Barcelona', 'Dublin', 'Brussels', 'Hamburg', 'Copenhagen', 'Berlin',
'Budapest', 'Bucharest']

Worst solution:  8346.94  km
['Hamburg', 'Brussels', 'Barcelona', 'Dublin', 'Copenhagen', 'Bucharest', 'Istanbul', 'Belgrade',
'Budapest', 'Berlin']

Mean solution:  7559.54  km
Standard deviation:  195.16  km



---- Number of cities:  24  ----
Best solution:  12690.64  km
['Paris', 'Brussels', 'Munich', 'Vienna', 'Prague', 'Berlin', 'Hamburg', 'Copenhagen', 'Stockholm',
'Saint Petersburg', 'Moscow', 'Kyiv', 'Warsaw', 'Budapest', 'Belgrade', 'Bucharest', 'Istanbul',
'Sofia', 'Rome', 'Milan', 'Barcelona', 'Madrid', 'Dublin', 'London']

Worst solution:  18470.14  km
['Istanbul', 'Rome', 'Milan', 'Berlin', 'Hamburg', 'Copenhagen', 'Warsaw', 'Bucharest', 'Sofia',
'London', 'Dublin', 'Prague', 'Vienna', 'Budapest', 'Belgrade', 'Munich', 'Brussels', 'Paris',
'Madrid', 'Barcelona', 'Stockholm', 'Saint Petersburg', 'Moscow', 'Kyiv']

Mean solution:  14343.76  km
Standard deviation:  1133.76  km
```

As observed in the output results, after 20 runs with random initial solutions, the standard deviation for 6 cities is 195.16km, which is low, indicating that with enough runs it can find the best solution (since the exhaustive search also has a similar best solution). However, the standard deviation for 24 cities is 1133.76km, which is much larger and less likely to find the best solution.

and the best and worst plan for 10 and 24 cities:

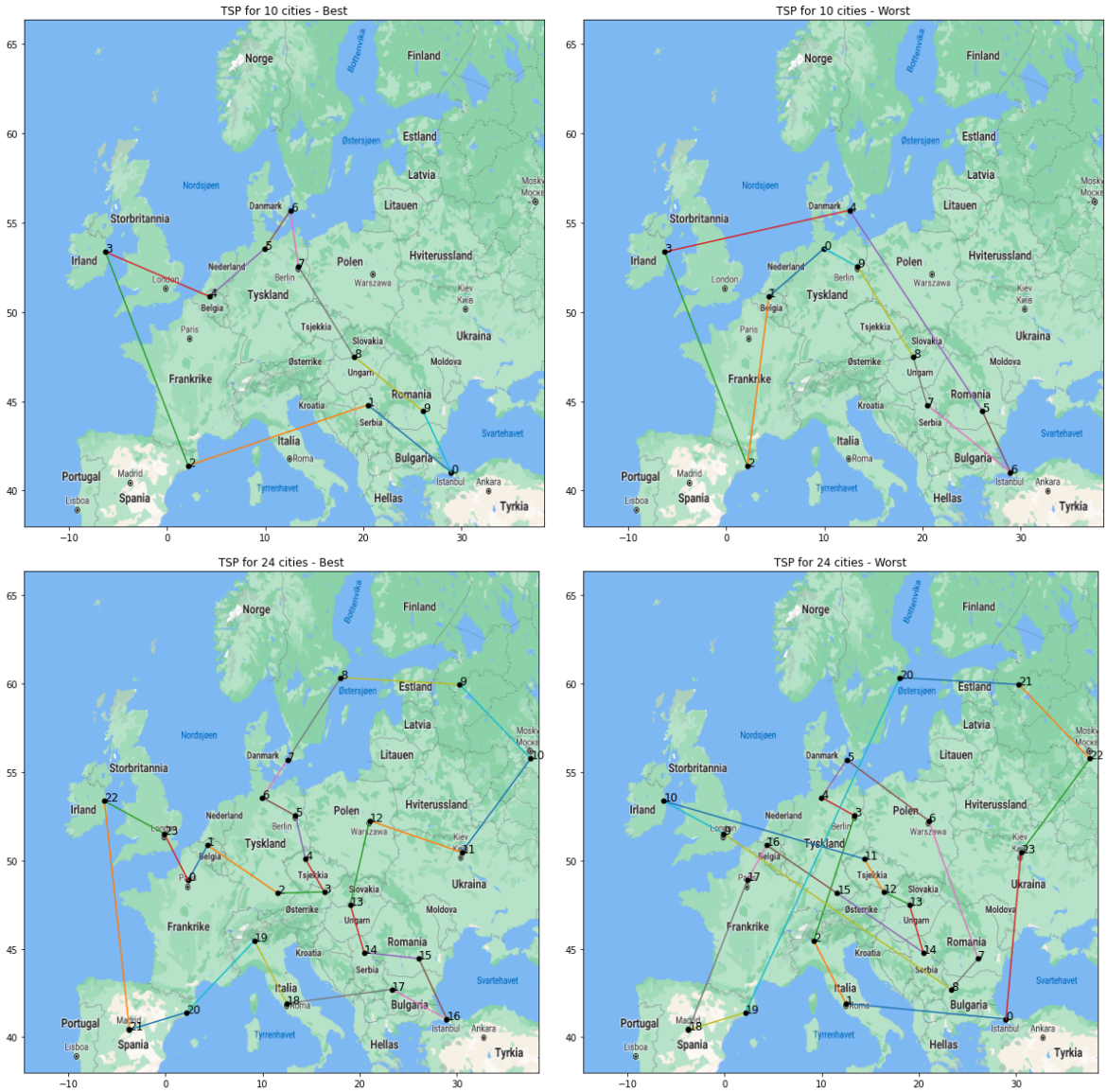

Figure 3: Hill climbing results for 10 cities. Left is best plan and right is worst plan

The following results are from line 82-131 of **IN3050_Oblig1.py**.

4 Genetic algorithm

A Genetic Algorithm (GA) is a type of Evolutionary Algorithm (EA) that incorporates mutation and crossover as operators. By doing so it incorporates a mix of exploration and exploitation. Exploration is the diversity, while exploitation is looking at the best candidates.

First, we need to choose mutation and crossover operators appropriate for the problem. Secondly, we need to choose the size of our population. Then, by following the scheme for general EA, we evaluate and choose better candidates for our population in the next generation. We repeat this until a termination condition is satisfied, which means that we need to define the termination condition.

4.1 Mutation

There are several mutation operators to choose from. One of them is inversion mutation. Since the TSP-problem is an adjacency problem, maintaining most of the original structure without breaking too many links is important, hence inversion mutation. This is similar to how evolution works where a sequence of DNA strings with a slight alteration in it can cause mutation without breaking too many links. This method is explained in chapter 4.5 of Eiben and Smith textbook.

4.2 Crossover

In order to create offsprings that we can mutate, we need operators for creating these offsprings. To do so we can use crossover operators. Some familiar operators are Partially mapped crossover (PMX) and Edge crossover. Here, the chosen method is PMX. This is due to the fact that the offspring maintains a bit of "qualities" from both parents since it shares segments with both parents.

4.3 Selection

Selection will occur in two places: selection of parents and selection of survivors.

After initializing and evaluating a population, the next step is to choose parents. Parents can be chosen from a type of selection called tournament selection. In tournament selection, k random candidates is chosen and we choose the best of those candidates as parent. This goes on until there are no more candidates to choose from current population. In other words, the number of parents you get by this method is

$$\text{number of parents} = \left\lceil \frac{\text{size of population}}{k} \right\rceil$$

the number k may vary, but should not be too big as it may eliminate potential candidates as parents. Too small k can lead to more exploration, but at the cost of computational power. Thus, choosing $k = 5$ seems reasonable.

Then, by choosing two parents at the time we make two offsprings. If there is only one parent left we are done. This ensures that each parent gets paired. In other words, the number of offsprings is

$$\text{number of offsprings} = \left\lfloor \frac{\text{number of parents}}{k} \right\rfloor$$

Next, we need to choose which candidates from previous generation of population and current offsprings we want in our next generation population. Since the population size remains the same, we have survivor selection. The method chosen here is an elitist strategy called $(\mu + \lambda)$-selection. This method chooses the best candidates by looking at the set of previous population and children, where μ is population size and λ offspring size. By using this method we choose a majority of candidates that performs well, but can disregard worse candidates without regards for potentially better offsprings. To find the μ best candidates we need to sort the candidates, and the choice of algorithm used is quicksort **(1)** .

4.4 Population size

In order to get a variation of exploitation and exploration, the choice of population size matters. Too small of a population doesn't utilizes exploration enough, thus potentially never leaving local maxima to find potentially better solution, whereas too big of a population might be unnecessary due to solution convergence (as seen in figure) and higher computational cost. Thus, the choices of three different is 100, 200 and 400.

4.5 Fitness

The best candidates is chosen based on their fitness. Here, the fitness function is simply the total tour distance, which means that the best fitness is the smallest fitness value. This choice is purely due to simplicity and laziness from the authors part. The only difference is that the average fitness plot will be inverted, with convergence being at the lowest tour distance.

4.6 Termination condition

Lastly, we need termination conditions when certain criteria is met. Some examples of termination conditions can be as following:
- reaching some (known/hoped) fitness
- reaching some maximum allowed number of generations
- reaching some minimum level of diversity
- reaching some specified number of generations without fitness improvement

Since we don't know anything about expected fitness, we can't set any criteria here. To terminate based on the fact that we have reached certain generations without improvement or have minimum level of diversity reduces exploration of potentially better solution in future generations (due to mutation), so the only termination condition that balances exploration and exploitation is the maximum allowed number of generations. Here, the chosen number of maximum generations is 400.

4.7 Results from Genetic Algorithm

Similarly to Hill climbing, for all three population sizes we report the best, worst, mean and standard deviation of tour distance out of 20 runs. In addition, we also measure the average time for the genetic

algorithm and an average fitness plot across each generation to observe it's performance.

Here are the output results after 20 runs for both 10 and 24 cities and the population sizes 100, 200 and 400 and 400 generations:

```
    >>> IN3050_Oblig1.py

Reloaded modules: IN3050_Oblig1_functions
Genetic Algorithm
400 generations

---- Number of cities:  10  ----

--- Population:  100  ---
Runtime (ns):  175853025.0
Best solution:  7486.309999999999  km
['Hamburg', 'Brussels', 'Dublin', 'Barcelona', 'Belgrade', 'Istanbul', 'Bucharest', 'Budapest',
'Berlin', 'Copenhagen']

Worst solution:  7486.3099999999995  km
['Belgrade', 'Istanbul', 'Bucharest', 'Budapest', 'Berlin', 'Copenhagen', 'Hamburg', 'Brussels',
'Dublin', 'Barcelona']

Mean solution:  7486.31  km
Standard deviation:  0.0  km



--- Population:  200  ---
Runtime (ns):  511899525.0
Best solution:  7486.309999999999  km
['Hamburg', 'Brussels', 'Dublin', 'Barcelona', 'Belgrade', 'Istanbul', 'Bucharest', 'Budapest',
'Berlin', 'Copenhagen']

Worst solution:  7486.309999999999  km
['Hamburg', 'Brussels', 'Dublin', 'Barcelona', 'Belgrade', 'Istanbul', 'Bucharest', 'Budapest',
'Berlin', 'Copenhagen']

Mean solution:  7486.31  km
Standard deviation:  0.0  km



--- Population:  400  ---
Runtime (ns):  1727791800.0
Best solution:  7486.309999999999  km
['Hamburg', 'Brussels', 'Dublin', 'Barcelona', 'Belgrade', 'Istanbul', 'Bucharest', 'Budapest',
'Berlin', 'Copenhagen']

Worst solution:  7486.309999999999  km
```

```
['Hamburg', 'Brussels', 'Dublin', 'Barcelona', 'Belgrade', 'Istanbul', 'Bucharest', 'Budapest',
'Berlin', 'Copenhagen']

Mean solution:  7486.31  km
Standard deviation:  0.0  km




---- Number of cities:  24  ----

--- Population:  100  ---
Runtime (ns):  166858050.0
Best solution:  12325.930000000002  km
['Saint Petersburg', 'Moscow', 'Kyiv', 'Bucharest', 'Istanbul', 'Sofia', 'Belgrade', 'Budapest',
'Vienna', 'Warsaw', 'Prague', 'Munich', 'Milan', 'Rome', 'Barcelona', 'Madrid', 'Dublin', 'London',
'Paris', 'Brussels', 'Hamburg', 'Berlin', 'Copenhagen', 'Stockholm']

Worst solution:  12846.210000000001  km
['Bucharest', 'Kyiv', 'Moscow', 'Saint Petersburg', 'Stockholm', 'Copenhagen', 'Berlin', 'Warsaw',
'Budapest', 'Vienna', 'Prague', 'Munich', 'Hamburg', 'Brussels', 'Paris', 'London', 'Dublin',
'Madrid', 'Barcelona', 'Milan', 'Rome', 'Belgrade', 'Sofia', 'Istanbul']

Mean solution:  12550.18  km
Standard deviation:  152.16  km



--- Population:  200  ---
Runtime (ns):  416102205.0
Best solution:  12287.070000000002  km
['Warsaw', 'Vienna', 'Budapest', 'Belgrade', 'Sofia', 'Istanbul', 'Bucharest', 'Kyiv', 'Moscow',
'Saint Petersburg', 'Stockholm', 'Copenhagen', 'Hamburg', 'Brussels', 'Paris', 'London', 'Dublin',
'Madrid', 'Barcelona', 'Rome', 'Milan', 'Munich', 'Prague', 'Berlin']

Worst solution:  12805.670000000002  km
['Budapest', 'Belgrade', 'Bucharest', 'Istanbul', 'Sofia', 'Rome', 'Milan', 'Barcelona', 'Madrid',
'Dublin', 'London', 'Brussels', 'Paris', 'Munich', 'Berlin', 'Hamburg', 'Copenhagen', 'Stockholm',
'Saint Petersburg', 'Moscow', 'Kyiv', 'Warsaw', 'Prague', 'Vienna']

Mean solution:  12485.31  km
Standard deviation:  151.11  km



--- Population:  400  ---
Runtime (ns):  1110754375.0
Best solution:  12287.069999999998  km
['Paris', 'Brussels', 'Hamburg', 'Copenhagen', 'Stockholm', 'Saint Petersburg', 'Moscow', 'Kyiv',
'Bucharest', 'Istanbul', 'Sofia', 'Belgrade', 'Budapest', 'Vienna', 'Warsaw', 'Berlin', 'Prague',
'Munich', 'Milan', 'Rome', 'Barcelona', 'Madrid', 'Dublin', 'London']
```

```
Worst solution:  12775.610000000004  km
['Budapest', 'Vienna', 'Munich', 'Prague', 'Berlin', 'Copenhagen', 'Hamburg', 'Brussels', 'London',
'Dublin', 'Paris', 'Madrid', 'Barcelona', 'Milan', 'Rome', 'Belgrade', 'Sofia', 'Istanbul',
'Bucharest', 'Kyiv', 'Moscow', 'Saint Petersburg', 'Stockholm', 'Warsaw']

Mean solution:  12378.65  km
Standard deviation:  113.48  km
```

and the plots

Results of the optimized tour for 10 cities and three different population sizes

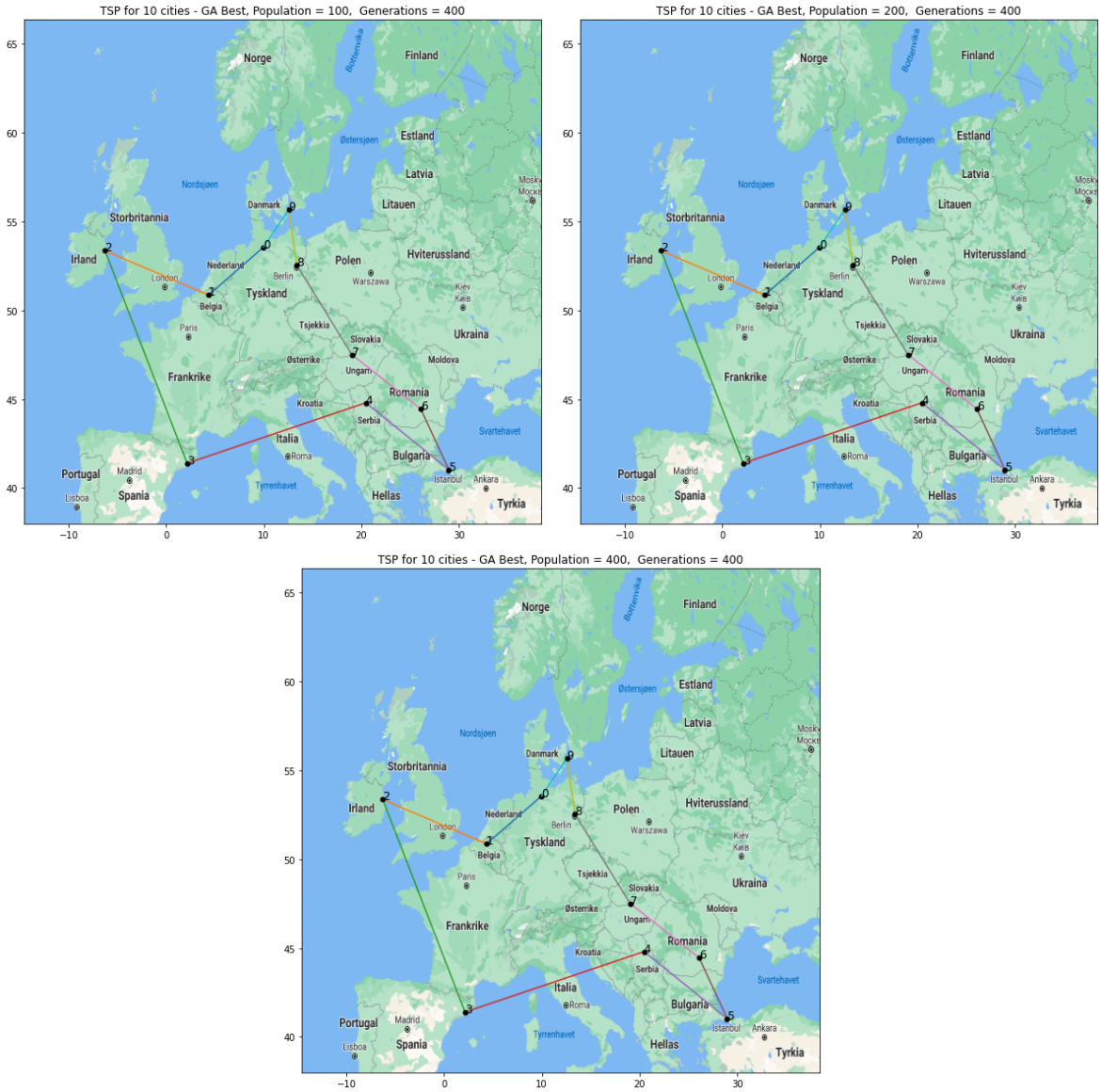

Figure 4: Genetic Algorithm results for 10 cities

Results of the optimized tour for 24 cities and three different population sizes

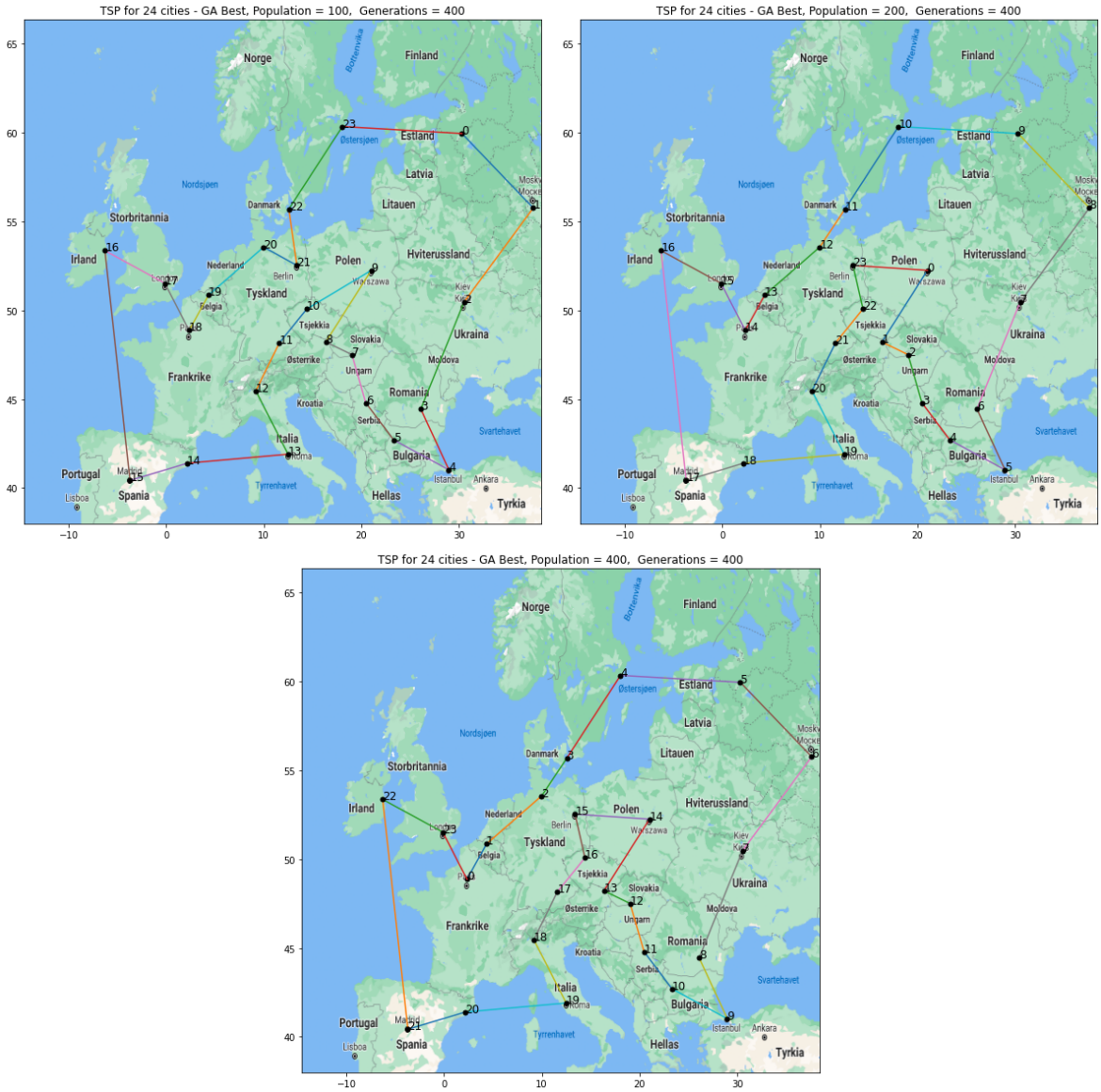

Figure 5: Genetic Algorithm results for 24 cities

And it's performances for each city with three different population sizes

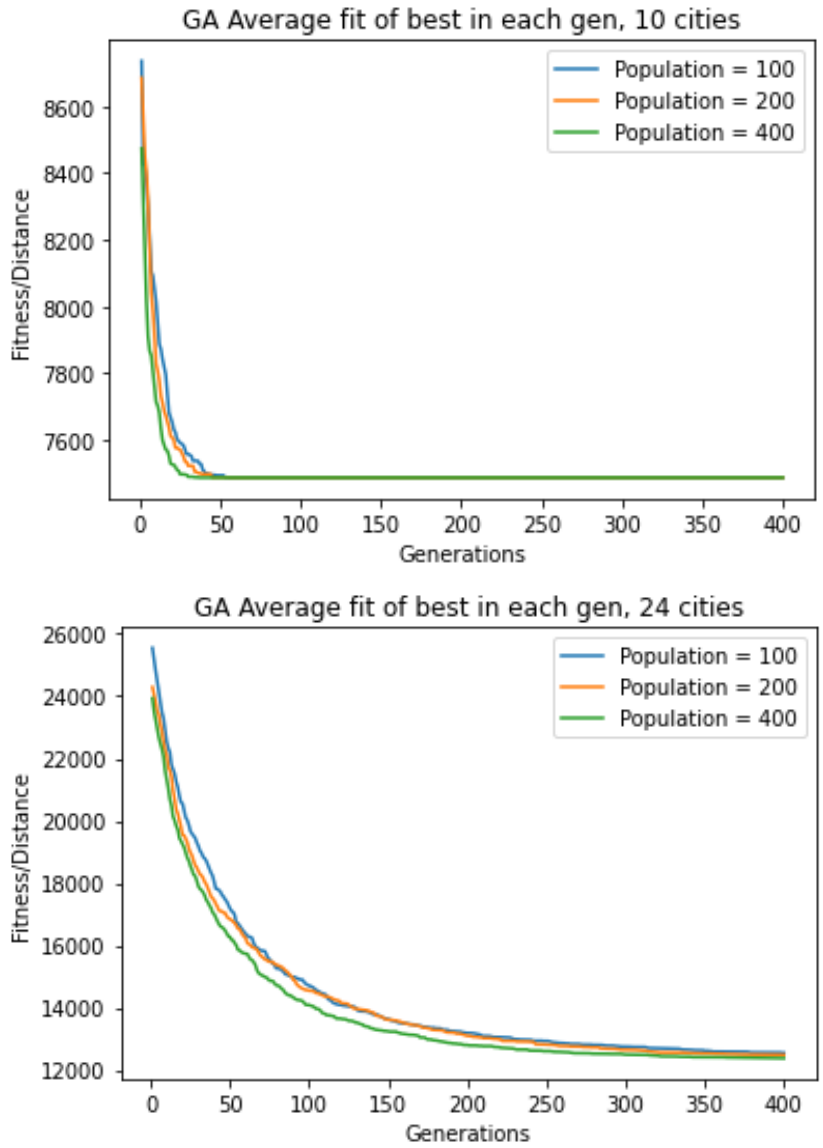

Figure 6: Genetic Algorithm fitness performance. Upper plot is for $n = 10$ while lower plot is for $n = 24$

As observed, for both 10 and 24 cities the largest population not only converges faster (although a small difference), but also finds the better optimized tour.

The following results are from line 140-220 of **IN3050_Oblig1.py**.

4.8 Observations

For 10 cities, the GA indeed managed to find the shortest tour for all three population sizes (ignoring the floating points).
As for the running time for both 10 and 24 cities, it performed very well. To compare, Exhaustive search spent an average 3.5 seconds on 10 cities while GA spent an average of 1.7 seconds (after ns conversion) with it's largest population. Similar observations and performance can be seen for 24 cities, which is astounding.

In terms of the mean tour distance and standard deviation for 24 cities when compared to Hill climbing, GA performs very well as observed. It has better mean distance and lower standard deviation.

While Exhaustive search have to inspect $n!$ tours to find it's best solution, the optimal solution found by GA does not require such criteria. The total amount of tours inspected by GA depends on the number of population, parents and offspring as well as the maximum allowed generations. Let μ, ρ and λ be the population size, number of parents and number of offsprings respectively, and let G be the maximum number of generations. Secondly, we need to define what inspecting a tour means. Here, it would simply mean to calculate distance and comparison. Then we have the following:
1) Evaluate population
2) Choose parents by tournament selection, with k selected at a time
3) Evaluate the mutated offsprings
4) By survivor selection, choose the best from both previous population and offspring

In step 1) we inspect μ tours. In step 2) we again inspect μ tours when selecting parents (due to comparison). In step 3) we evaluate the mutated offsprings by inspecting λ tours (since PMX crossover and inversion mutation doesn't inspect the tour). Lastly, in step 4) we have survivor selection. Since we first need to sort the combined set of previous population and offsprings we have to inspect this combined set, thus inspecting $\mu + \lambda$ tours. Then for each generation steps 2) - 4) are repeated, and adding the initial population in step 1), the total amount of inspected tours $\mathbf{I}$ becomes

$$\mathbf{I} = \mu + 2G(\mu + \lambda)$$

for

$$\rho = \left\lceil \frac{\mu}{k} \right\rceil \qquad \lambda = \left\lfloor \frac{\rho}{2} \right\rfloor \cdot 2$$

thus, for 24 cities, $G = 400$, $\mu = 400$ and $k = 5$ we get $\mathbf{I} = 384400$ inspected tours by using the GA, which is far more feasible than exhaustive search.

The GA performs very well in terms of finding optimal solutions.

5 References and files

List of relevant codes and files.

Codes:
IN3050_Oblig1_functions.py
IN3050_Oblig1.py

Relevant plots (from the map **plots**):

Figure 1:
Exhaustive search time plot.png

Figure 2:
Exhaustive search TSP_6.png
Exhaustive search TSP_10.png

Figure 3:
Hill climbing TSP_10 best.png
Hill climbing TSP_10 worst.png
Hill climbing TSP_24 best.png
Hill climbing TSP_24 worst.png

Figure 4:
GA TSP_10 best population=100.png
GA TSP_10 best population=200.png
GA TSP_10 best population=400.png

Figure 5:
GA TSP_24 best population=100.png
GA TSP_24 best population=200.png
GA TSP_24 best population=400.png

Figure 6:
GA TSP_10 Best fitness plot
GA TSP_24 Best fitness plot

References:
(1): https://www.uio.no/studier/emner/matnat/ifi/IN2010/h24/undervisningsmateriale/in2010_dark_handout.pdf